Quantum-Safe Hybrid Key Exchanges with KEM-Based Authentication

Christopher Battarbee[1], Christoph Striecks[2], Ludovic Perret[1,4], Sebastian Ramacher[2], and Kevin Verhaeghe[3⋆]

[1] Sorbonne University, CNRS, LIP6 F-75005 Paris, France
[2] AIT Austrian Institute of Technology, Vienna, Austria
[3] ETH Zurich, Zurich, Switzerland
[4] Laboratoire de Recherche de l'EPITA, Le Kremlin-Bicêtre, France

Abstract. Authenticated Key Exchange (AKE) between any two entities is one of the most important security protocols available for securing our digital networks and infrastructures. In PQCrypto 2023, Bruckner, Ramacher and Striecks proposed a novel hybrid AKE (HAKE) protocol dubbed Muckle+ that is particularly useful in large quantum-safe networks consisting of a large number of nodes. Their protocol is hybrid in the sense that it allows key material from conventional, post-quantum, and quantum cryptography primitives to be incorporated into a single end-to-end authenticated shared key.
To achieve the desired authentication properties, Muckle+ utilizes post-quantum digital signatures. However, available instantiations of such signatures schemes are not yet efficient enough compared to their post-quantum key-encapsulation mechanism (KEM) counterparts, particularly in large networks with potentially several connections in a short period of time. To mitigate this gap, we propose Muckle# that pushes the efficiency boundaries of currently known HAKE constructions. Muckle# uses post-quantum key-encapsulating mechanisms for implicit authentication inspired by recent works done in the area of Transport Layer Security (TLS) protocols, particularly, in KEMTLS (CCS'20).
We port those ideas to the HAKE framework and develop novel proof techniques on the way. Due to our KEM-based approach, the resulting protocol has a slightly different message flow compared to prior work that we carefully align with the HAKE framework and which makes our changes to Muckle+ non-trivial.

Keywords: hybrid authenticated key exchange, post-quantum cryptography, quantum cryptography

1 Introduction

The continuous progress of quantum technologies is making the deployment of global quantum-safe infrastructures an increasingly pressing matter. Perhaps most visibly in this direction, the first post-quantum cryptographic standards were formally published by the National Institute of Standards and Technology (NIST)[5] in 2024, following a multi-year and massive effort from a large community of researchers. Indeed, these quantum-resistant, or "post-quantum", technologies are entering a stage of maturity whereby their deployment into existing network architectures is a growing field of study.

In practice, the new post-quantum standards are expected to be deployed in tandem with existing techniques from the field of *Authenticated* Key Exchanges (AKEs) [Mau93,BR95]. In a public network of peers, for example, all the security guarantees of a post-quantum key encapsulation mechanism (KEM) are void if one party can impersonate the other, so we seek to guarantee that any two parties in the network can verify each other's authenticity – such a guarantee is called end-to-end authenticity. In an AKE protocol, we seek to establish a session key while achieving end-to-end authenticity.

⋆ Work done while with AIT.
[5] https://csrc.nist.gov/projects/post-quantum-cryptography

Motivation. There are several techniques by which one can achieve end-to-end authenticity, including using some pre-shared keying material (PSK). However, it is a well-known result that in a network of n peers one requires $\mathcal{O}(n^2)$ initial PSK distributions, so this type of authentication might not be suitable in a large or dynamic network scenario, particularly if we wish to be able to add new peers to the network.

In such a scenario, it may be more appropriate to use certificate-based authentication. Here, we suppose that the authentication is based on asymmetric cryptography, and some third-party, trusted certificate authority (CA) guarantees the authenticity of each peer's public keys. In turn, these asymmetric cryptography protocols are used, in some capacity, during an AKE protocol, establishing the authenticity of each party.

Typically, a digital signature scheme is utilized. One of the more famous examples of an AKE protocol is the Transport Layer Security (TLS) 1.3 [Res18] protocol where two parties run an unauthenticated ephemeral Diffie-Hellman-style handshake, and then sign various parts of the transcript, yielding authenticity. Several projects and initiatives are working on migrating network protocols such as TLS 1.3 to the post-quantum setting, both for KEMs and signatures, and evaluating their efficiency.[6] The currently available post-quantum standards [SAB+22,LDK+22,PFH+22,HBD+22] are such that the KEMs, in contrast to the pre-quantum setting, are more efficient than the currently available post-quantum signatures. In the post-quantum setting, therefore, it is tempting to also consider using KEMs as an authentication mechanism; particularly following the seminal approach taken by Schwabe, Stebila, and Wiggers on KEMTLS in [SSW20a] (which itself is based on the OPTLS proposal by Krawczyk and Wee [KW16]).

Towards defense-in-depth approaches. In parallel to the post-quantum standardization effort, the coming quantum threat has also motivated the deployment of large QKD-based testbeds; notably in China[7], in the EU with the EuroQCI network[8] and the various testbeds distributed throughout Europe [RRL+19,MBO+23,BVB+24], and in the UK with the Quantum Hub[9]. Whilst the first motivation for such testbeds was arguably to increase the maturity of QKD devices, the next step is to move towards a practical QKD-based network in real-life use-cases. This leads to the problem of integrating QKD with existing security protocols, and to the study of the combination (or, hybridization) of QKD with modern cryptography. Particularly, the European Commission recommends PQC in hybridization with currently deployed cryptographic primitives or QKD.[10]

Recently, hybrid AKE (HAKE) [DHP20,BRS23] has emerged as a so-called "defense-in-depth" solution to these problems, incorporating key material from conventional and PQC primitives, as well as from QKD. The overall goal is resilience: despite the intensive research into their security, the post-quantum standards are young algorithms whose failure is not unprecedented; but the promised unconditional security of QKD is caveated by the relatively immature devices on which current QKD is implemented. One aims, therefore, to utilize multiple sources of key material, in such a way that if all but one of these key-material sources fail, the resulting shared key between any two parties is still authentic and confidential.

In a different context, KEM combiners [GHP18] were proposed to enhance the security of combined key material from different KEMs on the primitive level. On the protocol level, we want to achieve more in terms of authentication and forward security which makes HAKE approaches the most suitable ones.

[6] E.g., https://blog.cloudflare.com/pq-2024/, https://www.microsoft.com/en-us/research/project/post-quantum-tls/, https://security.googleblog.com/2024/08/post-quantum-cryptography-standards.html

[7] https://physicsworld.com/a/quantum-cryptography-network-spans-4600-km-in-china/

[8] https://digital-strategy.ec.europa.eu/en/policies/european-quantum-communication-infrastructure-euroqci

[9] https://uknqt.ukri.org/success-stories/uk-quantum-networks/

[10] https://digital-strategy.ec.europa.eu/en/news/commission-publishes-recommendation-post-quantum-cryptography

Forward security. As well as tolerating "real-time" faults, protocols in the HAKE setting naturally have a higher degree of forward security, in the following sense. Forward security is an essential security features in nowadays protocols as it guarantees the security of past protocol sessions even in case of key leakage in the current session. Such features has been demonstrated to be of significant interest in interactive key-exchange protocols [Gün90,DvOW92,DDG+20,RSS23], public-key encryption [CHK03,Gro21], digital signatures [BM99,DGNW20], search on encrypted data [BMO17], 0-RTT key exchange [GHJL17,DJSS18,CRSS20,DGJ+21], updatable cryptography [SS23], mobile Cloud backups [DCM20], proxy cryptography [DKL+18], Tor [LGM+20], and content-delivery networks [DRSS21], among others.

In our AKE setting, consider the security model in which the security proof of TLS 1.3 is executed [DFGS21]; one does not model for the scenario in which the ephemeral secret established by a Diffie-Hellman [DH76] handshake is compromised after the session has accepted. Of course, such a compromise would completely reveal all secrets derived by that session of TLS 1.3, so there is a sense in which the secret established by the ephemeral Diffie-Hellman handshake has to remain secret "forever." Obviously this is not realistic in a post-quantum setting; the problem can in part be handled by updating authenticated key-exchange mechanisms to use post-quantum ephemeral handshakes (probably via a post-quantum KEM). On the other hand, a scenario in which all confidence is rapidly lost in a highly-regarded KEM candidate is not unprecedented, as seen with the SIDH break [CD23,Rob23,MMP+23].

Switching to the HAKE model improves forward security here, because as well as mixing three sources of secrets in such a way that any two can fail at the same time, the QKD link is resistant to the store-now-decrypt-later attacks relevant to both classical and post-quantum KEMs used for the ephemeral confidentiality part (whereby assuming of course working mechanisms for end-to-end authentication). This is essentially a consequence of the no-cloning theorem; one cannot directly copy the quantum states being exchanged. Thus, to learn any information about the secret being exchanged over a QKD link, one can only measure before the protocol is complete, and there is no "transcript" to attack passively after that point.

The HAKE framework. A HAKE framework is put forward in the pioneering work due to Dowling, Brandt Hansen, and Paterson [DHP20], in which the Muckle protocol is proposed. Muckle is TLS-like, in the sense that unauthenticated handshakes are retrospectively authenticated when the transcript is authenticated. However, the authors argue that the use-cases they target are such that a PSK infrastructure is appropriate to achieve end-to-end authentication, and so, as we have discussed, their protocol is not an appropriate choice for a large-scale and dynamic network. As an enhancement, Bruckner, Ramacher and Striecks [BRS23] proposed Muckle+, to efficiently derive an authenticated shared key even in large-scale quantum-safe networks. Muckle+ utilizes post-quantum signatures as the authentication mechanism. As we have discussed, currently known post-quantum signature schemes are not as efficient as post-quantum KEMs. Bruckner et al. already recognized such a property and left it as open problem to come up with more efficient authentication within HAKE.

In this work, we address this gap, and propose Muckle#, a protocol that solely utilizes post-quantum KEMs for authentication. In contrast with the signature case, only implicit authentication can be achieved directly with KEMs, and further rounds of message-authentication code (MAC) tags on the transcript of the protocol must be exchanged to yield explicit authentication. We provide a security proof of Muckle# within the HAKE framework.

Related work. The two works most closely related to this paper are [DHP20] and [BRS23], in which the Muckle and Muckle+ protocols are put forward, respectively. Crucially, in [DHP20], the HAKE security model is defined, and it is in this model that the security of Muckle, Muckle+ and Muckle# is proved. This model is a natural, hybrid-setting successor to the Bellare-Rogaway-style AKE models first put forward in [BR93].

Our protocol can be thought of as a synthesis of the Muckle+ protocol and the KEMTLS protocol defined in [SSW20a]. This latter work, in turn, is an improvement on an earlier attempt to achieve signature-free AKE [KW16]. We note also that, on the technical side, we make heavy

use of the arguments in the security proof of the KEMTLS protocol, in particular relying on the standard identical-until-bad techniques of [BR04].

There are various papers in the literature which seek to combine PQC and QKD. Mosca et al. [MSU13] define a protocol in which the authentication channel required for QKD is achieved with a post-quantum signature. They prove the security of this protocol in a different hybrid-setting AKE model, which is set-up to deal with quantum information (rather than treating the QKD link as a black-box, as in our case). The protocol does not mix different secret sources, as in the Muckle-related protocols. An experimental implementation of this kind of approach was achieved by Wang et al. in [WZW+21]. Closer to the spirit of our own work, a recent paper proposes the "Muckle++" protocol [GPH+24], which offers similar large-network flexibility to Muckle+ by including a signature authentication mode. A security proof is not provided, but the authors achieve an experimental implementation of their protocol on commercial hardware.

Contribution. Our contribution can be summarized as follows. We carefully construct a HAKE protocol, dubbed Muckle#, in the framework of HAKE that uses KEM-based authentication instead of a signature-based one. Along the way, we propose adapted proof steps due to absence of digital signatures for authentication compared to Muckle+. Our new protocol has a slightly different message flow that we carefully align with the HAKE framework and which makes our changes to the Muckle+ non-trivial.

1.1 More on the Technical Details

In the following, we discuss the technical details related to the security proof, modes of authentication, and modeling the QKD link in a more depth.

Security proof. We prove the security of Muckle# within the HAKE model, making repeated use of the arguments put forward in the security proof of KEMTLS. There are two main obstacles to overcome: fitting the KEMTLS arguments within a HAKE framework, and addressing the gap between implicit and explicit authentication that is introduced when one uses a KEM to authenticate. In Section 3.1, we discuss how these technicalities are addressed.

Modes of authentication. We briefly explain the difference in the authentication mechanisms of Muckle, Muckle+ and Muckle#. First, note that all three of those protocols begin with an unauthenticated exchange of ephemeral secrets; first of a classical KEM, then of a post-quantum KEM, and finally the exchange of a secret via a QKD protocol. Eventually, one uses a series of pseudo-random function evaluations to derive a handshake secret. The authentication mechanisms then differ thus:

- In the Muckle protocol it is assumed that any two parties in the network have already established a pre-shared key. They can then use this key to compute MACs on the transcript of the values exchanged, thereby authenticating the handshake secret.
- In the Muckle+ protocol, after the handshake secret is derived, the parties send (encrypted) certificates, which contain static, authenticated signing keys for a post-quantum signature scheme. Signatures of the transcript of the protocol are then computed with respect to these signing keys. After a final exchange of MACs (as is standard in such cases; see [Kra03]), the handshake secret is authenticated.
- In our proposed Muckle#, after the handshake secret is defined, the same exchange of certificates takes place. This time, however, the certificates contain static, authenticated encapsulation keys for a post-quantum KEM. Each entity then encapsulates an additional secret to their partner's long term encapsulation key, and upon decapsulation the resulting secret is, again via the application of pseudo-random function evaluations, "baked in" to the key schedule. Here, the authenticity is intuitively a result of the authenticity of the encapsulation key; since only the entity with the corresponding decapsulation key could recover the appropriate secret and adds this to the key scheduling. Crucially, only *implicit* authentication is achieved here, and a final MAC exchange is required to make this authentication explicit.

Modelling the QKD link. As with the predecessors, Muckle [DHP20] and Muckle+ [BRS23], we model the QKD link out-of-band. More specifically, each entity has a black-box GetKey that gives them a key value k_q, and the mechanism by which this is achieved is abstracted to some QKD protocol running in the background. We model the potential failure of this QKD link by giving the adversary access to a CorruptQK query, that reveals the key k_q.

Now, all current QKD protocols require some form of auxiliary, symmetrically authenticated channel for their unconditional security to hold. Part of the motivation for moving away from the symmetric authentication of Muckle, to public-key methods, was to remove the reliance on pre-shared key networks, which scale inherently badly. As such, there is some tension between the modeling assumptions we have made about the QKD link, and the potential use cases of Muckle+ and Muckle# in more dynamic networks.

This gap is not an easy one to address. One of the reasons to model QKD out-of-band is to avoid invoking the quantum-physical type arguments often seen in contemporary security proofs of QKD protocols, and indeed, any method of addressing the gap will likely be required to feature such analysis. We consider this out of scope for our work, whose main purpose is to demonstrate efficiency gains with respect to Muckle+, while retaining provable security within the same framework.

KEM-Based Authentication and Public-Key Infrastructures. Available Public-Key Infrastructures (PKIs) will not be applicable in our setting due to certifying long-term signatures keys only whereas in Muckle# we would need certifactes that include long-term encapsulation keys. The issue appears already in KEM-TLS which limits the use of such a protocol for Internet usage.

However, since we are in a different setting and particularly motivated by the EuroQCI and related QKD-based networks, a novel PKI has to be developed alongside anyways to cope with the novel requirements in such quantum-safe networks (where such a new PKI could incorporate certifying long-term KEM keys as well in that case). Hence, we strongly believe that our Muckle# protocol would definitely be applicable in such anticipated networks.

2 Preliminaries

In this section, we briefly recall notions related to (hybrid) authenticated key exchanges.

Notation. Let $\kappa \in \mathbb{N}$ be the security parameter. For a finite set $\mathcal{S}$, we denote by $s \leftarrow \mathcal{S}$ the process of sampling an element s uniformly from $\mathcal{S}$. For an algorithm A, we let $y \leftarrow A(\kappa, x)$ denote the process of running A on input (κ, x) with access to uniformly random coins, and assigning the result to y. (We may omit explicit mention of the κ-input and assume that all algorithms take κ as input.) We say an algorithm A is probabilistic polynomial time (PPT) if the running time of A is polynomial in κ by a probabilistic Turing machine. An algorithm A is called quantum polynomial time (QPT) if it is a uniform family of quantum circuits with size polynomial in κ. A function f is called negligible if its absolute value is smaller than the inverse of any polynomial, for sufficiently large input values (i.e., if $\forall c \in \mathbb{N}\ \exists k_0\ \forall \kappa \geq k_0 : |f(\kappa)| < 1/\kappa^c$).

2.1 Cryptographic Primitives and Schemes

Definition 1 (Pseudo-Random Function). *Let $\mathcal{F}\colon \mathcal{S} \times D \to \mathsf{R}$ be a family of functions and let Γ be the set of all functions $D \to \mathsf{R}$. For a PPT distinguisher $\mathcal{D}$ we define the advantage function as*

$$\mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{D},\mathcal{F}}(\kappa) = \left| \Pr\left[s \xleftarrow{R} \mathcal{S} : \mathcal{D}^{\mathcal{F}(s,\cdot)}(1^\kappa) = 1 \right] - \Pr\left[f \xleftarrow{R} \Gamma : \mathcal{D}^{f(\cdot)}(1^\kappa) = 1 \right] \right|.$$

$\mathcal{F}$ is a pseudorandom function (family) if it is efficiently computable and for all PPT distinguishers $\mathcal{D}$ there exists a negligible function $\varepsilon(\cdot)$ such that

$$\mathsf{Adv}^{\mathsf{PRF}}_{\mathcal{D},\mathcal{F}}(\kappa) \leq \varepsilon(\kappa).$$

A PRF $\mathcal{F}$ is a dual PRF [BL15], if $\mathcal{G} : D \times \mathcal{S} \to \mathsf{R}$ defined as $\mathcal{G}(d, s) = \mathcal{F}(s, d)$ is also a PRF.

We recall the notion of message authentication codes (MACs) as well as digital signature schemes, and the standard unforgeability notions below.

Definition 2 (Message Authentication Codes). *A message authentication code* MAC *is a triple* $(\mathsf{KGen}, \mathsf{Sign}, \mathsf{Ver})$ *of PPT algorithms, which are defined as:*

$\mathsf{KGen}(1^\kappa)$: *This algorithm takes a security parameter κ as input and outputs a secret key* sk.

$\mathsf{Auth}(\mathsf{sk}, m)$: *This algorithm takes a secret key $\mathsf{sk} \in \mathcal{K}$ and a $m \in \mathcal{M}$, and outputs an authentication tag τ.*

$\mathsf{Ver}(\mathsf{sk}, m, \tau)$: *This algorithm takes a secret key* sk*, a message $m \in \mathcal{M}$, and an authentication tag τ as input, and outputs a bit $b \in \{0, 1\}$.*

A MAC is correct if for all $\kappa \in \mathbb{N}$, for all $\mathsf{sk} \leftarrow \mathsf{KGen}(1^\kappa)$ and for all $m \in \mathcal{M}$, it holds that

$$\Pr\left[\mathsf{Ver}(\mathsf{sk}, m, \mathsf{Auth}(\mathsf{sk}, m)) = 1\right] = 1,$$

where the probability is taken over the random coins of KGen and Auth.

Definition 3 (EUF-CMA security of MAC). *For a PPT adversary $\mathcal{A}$, we define the advantage function in the sense of existential unforgeability under chosen message attacks (*EUF-CMA*) as*

$$\mathsf{Adv}^{\mathsf{euf\text{-}cma}}_{\mathcal{A},\mathsf{MAC}}(1^\kappa) = \Pr\left[\mathsf{Exp}^{\mathsf{euf\text{-}cma}}_{\mathcal{A},\mathsf{MAC}}(1^\kappa) = 1\right],$$

where the corresponding experiment is depicted in Experiment 1. If for all PPT adversaries $\mathcal{A}$ there is a negligible function $\varepsilon(\cdot)$ such that $\mathsf{Adv}^{\mathsf{euf\text{-}cma}}_{\mathcal{A},\mathsf{MAC}}(1^\kappa) \leq \varepsilon(\kappa)$, we say that MAC *is* EUF-CMA *secure.*

$\mathsf{Exp}^{\mathsf{euf\text{-}cma}}_{\mathcal{A},\mathsf{MAC}}(1^\kappa)$:
  $\mathsf{sk} \leftarrow \mathsf{KGen}(1^\kappa)$, $\mathcal{Q} \leftarrow \emptyset$
  $(m^*, \tau^*) \leftarrow \mathcal{A}^{\mathsf{Auth}', \mathsf{Ver}'}(1^\kappa)$
    where oracle $\mathsf{Auth}'(m)$:
      $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{m\}$
      return $\mathsf{Auth}(\mathsf{sk}, m)$
    where oracle $\mathsf{Ver}'(m, \tau)$:
      return $\mathsf{Ver}(\mathsf{sk}, m, \tau)$
  return 1, if $\mathsf{Ver}(\mathsf{sk}, m^*, \tau^*) = 1 \ \wedge \ m^* \notin \mathcal{Q}$, return 0, otherwise

Experiment 1: EUF-CMA security experiment for a MAC MAC.

We recall the notion of key-encapsulations mechanisms (KEMs), and the standard chosen-plaintext and chosen-ciphertext notions below.

Definition 4 (Key-Encapsulation Mechanism). *A key-encapsulation mechanism scheme* KEM *with key space $\mathcal{K}$ consists of the three PPT algorithms* $(\mathsf{KGen}, \mathsf{Enc}, \mathsf{Dec})$*:*

$\mathsf{KGen}(1^\kappa)$: *This algorithm takes a security parameter κ as input, and outputs public and secret keys* $(\mathsf{pk}, \mathsf{sk})$.

$\mathsf{Enc}(\mathsf{pk})$: *This algorithm takes a public key* pk *as input, outputs a ciphertext c and key K.*

$\mathsf{Dec}(\mathsf{sk}, c)$: *This algorithm takes a secret key* sk *and a ciphertext c as input, and outputs K or $\{\perp\}$.*

$\mathsf{Exp}^{\mathsf{ind}\text{-}T}_{\mathcal{A},\mathsf{KEM}}(\kappa)$:
  $(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KGen}(1^\kappa)$
  $(c^*, K_0) \leftarrow \mathsf{Enc}(\mathsf{pk}), K_1 \xleftarrow{R} \mathcal{K}$
  $\mathcal{Q} \leftarrow \emptyset$, $b \xleftarrow{R} \{0,1\}^\kappa$
  $b^* \leftarrow \mathcal{A}^{\mathcal{O}}(\mathsf{pk}, c^*, K_b)$
    where $\mathcal{O} = \{\mathsf{Dec}'\}$ if $T = \mathsf{cca}$ with oracle $\mathsf{Dec}'(c)$:
      $\mathcal{Q} \leftarrow \mathcal{Q} \cup \{c\}$
      return $\mathsf{Dec}(\mathsf{sk}, c)$
  return 1, if $b = b^* \ \wedge \ c^* \notin \mathcal{Q}$, return otherwise 0

Experiment 2: IND-CPA and IND-CCA security experiments for KEM with $T \in \{\mathsf{cpa}, \mathsf{cca}\}$.

We call a KEM correct if for all $\kappa \in \mathbb{N}$, for all $(\mathsf{pk},\mathsf{sk}) \leftarrow \mathsf{KGen}(\kappa)$, for all $(c, K) \leftarrow \mathsf{Enc}(\mathsf{pk})$, we have that

$$\Pr[\mathsf{Dec}(\mathsf{sk}, c) = K] = 1,$$

where the probability is taken over the random coins of KGen and Enc.

Definition 5 (IND-CPA and IND-CCA security of KEM). *For a PPT adversary $\mathcal{A}$, we define the advantage function in the sense of indistinguishability under chosen-plaintext attacks (IND-CPA) and indistinguishability under chosen-ciphertexts attacks (IND-CCA) as*

$$\mathsf{Adv}^{\mathsf{ind\text{-}cpa}}_{\mathcal{A},\mathsf{KEM}}(1^\kappa) = \left| \Pr\left[\mathsf{Exp}^{\mathsf{ind\text{-}cpa}}_{\mathcal{A},\mathsf{KEM}}(1^\kappa) = 1\right] - \frac{1}{2} \right|, \text{ and}$$

$$\mathsf{Adv}^{\mathsf{ind\text{-}cca}}_{\mathcal{A},\mathsf{KEM}}(1^\kappa) = \left| \Pr\left[\mathsf{Exp}^{\mathsf{ind\text{-}cca}}_{\mathcal{A},\mathsf{KEM}}(1^\kappa) = 1\right] - \frac{1}{2} \right|$$

where the corresponding experiments are depicted in Experiment 2. If for all PPT adversaries $\mathcal{A}$ there is a negligible function $\varepsilon(\cdot)$ such that

$$\mathsf{Adv}^{\mathsf{ind\text{-}cpa}}_{\mathcal{A},\mathsf{KEM}}(1^\kappa) \leq \varepsilon(\kappa) \text{ or } \mathsf{Adv}^{\mathsf{ind\text{-}cca}}_{\mathcal{A},\mathsf{KEM}}(1^\kappa) \leq \varepsilon(\kappa),$$

then we say that KEM is IND-CPA or IND-CCA secure, respectively.

We further use the Authenticated Encryption with Associated Data (AEAD) scheme as defined in [RBBK01].

Remark 1. We can straightforwardly define post-quantum security for our definitions by requiring that the definitions hold against QPT adversaries.

2.2 Hybrid Authenticated Key Exchange

We recall the hybrid authenticated key exchange (HAKE) security model [DHP20,BRS23]. The HAKE security experiment $\mathsf{Exp}^{\mathsf{hake,clean}}_{\mathcal{A},\Pi,n_P,n_S,n_T}$ is described as in [DHP20, Fig. 5, App. C]. Here, we only recall the execution environment, adversarial interaction, and matching sessions.

Execution environment. We consider a set of n_P parties $P_1, \ldots, P_{n_P}$ which are able to run up to n_S sessions of a key-exchange protocol between them, where each session may consist of n_T different stages of the protocol[11]. A "stage" implicitly describes a pair of stages, one local to the

[11] Notice the terminology here is different to that in, say, the analysis of TLS 1.3 [DFGS21]. What we call a *stage* would in that context be called a *session*, and we do not have any subdivision below the session level.

initiator and one local to the responder. A stage is said to have "accepted" if it has completed its key derivation schedule without aborting the protocol. Each party P_i has access to its long-term key pair $(\mathsf{pk}_i, \mathsf{sk}_i)$ and to the public keys of all other parties. Each session is described by a set of session parameters:

- $\rho \in \{\mathsf{init}, \mathsf{resp}\}$: The role (initiator or responder) of the party during the current session.
- $pid \in n_P$: The communication partner of the current session.
- $stid \in n_T$: The current stage of the session.
- $\alpha \in \{\mathsf{active}, \mathsf{accept}, \mathsf{reject}, \perp\}$: The status of the session. Initialized with $\perp$.
- $m_i[stid], i \in \{s, r\}$: All messages sent ($i = s$) or received ($i = r$) by a session up to the stage *stid*. Initialized with $\perp$.
- $k[stid]$: All session keys created up to stage *stid*. Initialized with $\perp$.
- $exk[stid], x \in \{q, c, s\}$: All ephemeral post-quantum (q), classical (c) or symmetric (s) secret keys created up to stage *stid*. Initialized with $\perp$.
- $pss[stid]$: The per-session secret state ($\mathsf{SecState}$) that is created during the stage *stid* for the use in the next stage.
- $st[stid]$: Storage for other states used by the session in each stage.

We describe the protocol as a set of algorithms $(f, \mathsf{KGen}XY, \mathsf{KGen}ZS)$:

- $f(\kappa, \mathsf{pk}_i, \mathsf{sk}_i, pskid_i, psk_i, \pi, m) \to (m', \pi')$: A probabilistic algorithm that represents an honest execution of the protocol. It takes a security parameter κ, the long-term keys $(\mathsf{pk}_i, \mathsf{sk}_i)$, the session parameters π representing the current state of the session, and a message m, and outputs a response m' and the updated session state π'.
- $\mathsf{KGen}XY(\kappa) \to (\mathsf{pk}, \mathsf{sk})$: A probabilistic asymmetric key-generation algorithm that takes a security parameter κ and creates a public-secret-key pair $(\mathsf{pk}, \mathsf{sk})$. $X \in \{E, L\}$ determines whether the created key is an ephemeral (E) or a long-term (L) secret key. $Y \in \{Q, C\}$ determines whether the key is classical (C) or post-quantum (Q).
- $\mathsf{KGen}ZS(\kappa) \to (psk, pskid)$: A probabilistic symmetric key-generation algorithm that takes a security parameter κ and outputs symmetric keying material (psk). $Z \in \{E, L\}$ determines whether the created key is an ephemeral (E) or a long-term (L) secret key.

For each party $P_1, \ldots, P_{n_P}$, classical as well as post-quantum long-term keys are created using the corresponding $\mathsf{KGen}XY$ algorithms. The challenger then queries a uniformly random bit $b \leftarrow \{0, 1\}$ that will determine the key returned by the Test query. From this point on, the adversary may interact with the challenger using the queries defined in the next section. At some point during the execution of the protocol, the adversary $\mathcal{A}$ may issue a single Test query[12] and present a guess for the value of b. If $\mathcal{A}$ guesses correctly and the session satisfies the cleanness predicate, the adversary wins the key-indistinguishability experiment.

Adversarial Interaction. The HAKE framework defines a range of queries that allow the attacker to interact with the communication:

- $\mathsf{Create}(i, j, role) \to \{(s), \perp\}$: Initializes a new session between party P_i with role *role* and the partner P_j. If the session already exists, then the query returns $\perp$, otherwise the session (s) is returned.
- $\mathsf{Send}(i, s, m) \to \{m', \perp\}$: Enables $\mathcal{A}$ to send messages to sessions and receive the response m' by running f for the session π_i^s. Returns $\perp$ if the session is not active.

[12] Unlike similar works in the area we explicitly limit our adversary to making exactly one Test query. The reason is for the sake of simplicity: proofs of protocols in which the adversary can issue many Test queries begin by guessing if a particular session was tested, and aborting otherwise, thereby allowing the proof to argue about a single tested session. By simply limiting to a single test query, the advantage derived in our proof will differ only by a constant factor from the advantaged derived for an equivalent proof in which multiple test queries were permitted. Since proofs of this kind are highly non-tight anyway, we consider this an acceptable sacrifice in the service of readability.

- $\mathsf{Reveal}(i, s, t)$: Provides $\mathcal{A}$ with the session keys, corresponding to a session π_i^s in stage t, if the session is in the accepted state. Otherwise, $\perp$ is returned.
- $\mathsf{Test}(i, s, t) \to \{k_b, \perp\}$: Provides $\mathcal{A}$ with the real (if $b = 1$) or random ($b = 0$) session key, for the session π_i^s in stage t, for the key-indistinguishably experiment.
- $\mathsf{Corrupt}XY(i) \to \{key, \perp\}$: Provides $\mathcal{A}$ with the long-term $XY \in \{\mathsf{SK}, \mathsf{QK}, \mathsf{CK}\}$ keys for P_i. If the key has been corrupted previously, then $\perp$ is returned. Specifically:
  - $\mathsf{CorruptSK}$: Reveals the long-term symmetric secret (if available).
  - $\mathsf{CorruptQK}$: Reveals the post-quantum long-term key (if available).
  - $\mathsf{CorruptCK}$: Reveals the classical long-term key (if available).
- $\mathsf{Compromise}XY(i, s, t) \to \{key, \perp\}$: Provides $\mathcal{A}$ with the ephemeral $XY \in \{\mathsf{QK}, \mathsf{CK}, \mathsf{SK}, \mathsf{SS}\}$ keys created during the session π_i^s prior to stage t. If the ephemeral key has already been compromised, then $\perp$ is returned. Specifically:
  - $\mathsf{CompromiseQK}$: Reveals the ephemeral post-quantum key.
  - $\mathsf{CompromiseCK}$: Reveals the ephemeral classical key.
  - $\mathsf{CompromiseSK}$: Reveals the ephemeral quantum key.
  - $\mathsf{CompromiseSS}$: Reveals the ephemeral per session state ($\mathsf{SecState}$).

Matching sessions. Furthermore, we recall the definitions of matching sessions [LKZC07] and origin sessions [CF12] which covers that the two parties involved in a session have the same view of their conversation.

Definition 6 (Matching sessions). *We consider two sessions π_i^s and π_j^r in stage t to be matching if all messages sent by the former session $\pi_i^s.m_s[t]$ match those received by the later $\pi_j^r.m_r[t]$ and all messages sent by the later session $\pi_j^r.m_s[t]$ are received by the former $\pi_i^s.m_r[t]$.*

π_i^s is considered to be prefix-matching with π_j^r if $\pi_i^s.m_s[t] = \pi_j^r.m_r[t]'$ where $\pi_j^r.m_r[t]$ is truncated to the length of $\pi_i^s.m_s[t]$ resulting in $\pi_j^r.m_r[t]'$.

Definition 7 (Origin sessions). *We consider a session π_i^s to have an origin session with π_j^r if π_i^s matches π_j^r or if π_i^s prefix-matches π_j^r.*

Implicit vs. explicit authentication. KEM authentication by itself achieves only "implicit authentication". Intuitively, the idea is that as long as KEM security holds and long-term secrets are not prematurely compromised, only the intended party can recover the intended secret. The authentication is implicit because, under these assumptions, any keying material derived from this implicitly authenticated secret must only be available to the intended party. We do not, however, get a guarantee that the intended party has indeed accessed this secret – so-called "explicit authentication". Unlike in the signature case, which provides authentication and proof that the intended party is "live" in the same step, this gap must be addressed. This is the reason for the MAC exchange at the end of the protocol – it provides a proof that the partner is 'live'. The strategy of the proof is to replace the secret that was encapsulated against the static public key, by a random secret, bounding the difference in advantage between these two games by the advantage in a $\mathsf{ind\text{-}cca1}$ game. We can then argue that a peer who accepted the final MAC tag without an honest partner session must have accepted a forgery, which allows us to bound the final advantage by an advantage in a MAC unforgeability game. Note that the case in which an honest session accepts the final MAC tag without an honest a partner is called "malicious acceptance" in [SSW20b].

HAKE security. Dowling et al. [DHP20] define key indistinguishability (i.e., what we dub HAKE security) with respect to a predicate clean. However, their predicate is specific to Muckle and, hence, we therefore only give the formal security definition next and postpone the discussion of the predicate to Section 3.1.

Definition 8 (HAKE security). *Let Π be a key-exchange protocol and $n_P, n_S, n_T \in \mathbb{N}$. For a predicate clean and an adversary $\mathcal{A}$, we define the advantage of $\mathcal{A}$ in the HAKE key-indistinguishability game as*

$$\mathsf{Adv}^{\mathsf{hake,clean}}_{\mathcal{A},\Pi,n_P,n_S,n_T}(\kappa) = \left|\Pr\left[\mathsf{Exp}^{\mathsf{hake,clean}}_{\mathcal{A},\Pi,n_P,n_S,n_T}(\kappa) = 1\right]\right|.$$

Table 1. Values for the contexts used in the Muckle# key schedule. The context inputs follow the choices in the TLS 1.3 handshake [DFGS21].

Label	Context Input	Label	Context Input
H_ε	""	H_0	$H(\text{""})$
H_1	$H(\boldsymbol{m_1} \| \boldsymbol{m_2})$	H_2	$H(\boldsymbol{m_1} \| \ldots \| \boldsymbol{m_4})$
H_3	$H(\boldsymbol{m_1} \| \ldots \| \boldsymbol{m_6})$	H_4	$H(\boldsymbol{m_1} \| \ldots \| \boldsymbol{m_7})$
H_5	$H(\boldsymbol{m_1} \| \ldots \| \boldsymbol{m_8})$		

Table 2. Values for the labels used in the Muckle+ key schedule for domain separation. Some of these labels are directly based on the corresponding labels in the TLS 1.3 handshake [DFGS21]. The concrete value of these labels is unimportant as long as they are unique.

Label	Label Input	Label	Label Input
ℓ_0	"derive k c"	ℓ_1	"derive k pq"
ℓ_2	"first ck"	ℓ_3	"second ck"
ℓ_4	"third ck"	ℓ_5	"fourth ck"
ℓ_6	"i hs traffic"	ℓ_7	"r hs traffic"
ℓ_8	"hs derived"	ℓ_9	"first ak"
ℓ_{10}	"i ahs traffic"	$\ell 11$	"r ahs traffic"
ℓ_{12}	"ahs derived"	ℓ_{13}	"second ak"
ℓ_{14}	"derive i fk"	ℓ_{15}	"derive r fk"
ℓ_{16}	"i app traffic"	ℓ_{17}	"r app traffic"
ℓ_{18}	"secstate"		

We say that Π is HAKE-secure if $\mathsf{Adv}^{\mathsf{hake,clean}}_{\mathcal{A},\Pi,n_P,n_S,n_T}(\kappa)$ is negligible in the security parameter κ for all $\mathcal{A}$.

If security also holds against any QPT adversary $\mathcal{A}$, then we call Π post-quantum secure.

3 Muckle#: A HAKE Protocol With KEM-Based Authentication

In this section, we present our novel variant Muckle#. To recall, the Muckle+ protocol [BRS23] combines conventional, PQC, and QKD keys through the use of a key derivation function. More concretely, Muckle+ requires conventional and post-quantum KEMs, a QKD mechanism as well as a digital signature scheme to create the final shared secret. Muckle# on the other side is similar to Muckle+, but utilizes post-quantum KEMs instead of signatures in the authentication step. Figure 1 depicts one stage of the Muckle# protocol that overall runs in several stages.

The protocol flow is as follows. The initiator and the responder in the initial stage possess secret decryption keys sk_I and sk_R of a post-quantum KEM, respectively, and a secret state $\mathsf{SecState}$ (which is initially set to an empty string).

The initiator and the responder further possess certificates $cert_I$ and $cert_R$, respectively, from a Public-Key Infrastructure (which is out of scope of this work). The certificates contain the verification keys pk_I and pk_R (available to all entities), respectively.

The initiator and the responder have access to the labels defined in Table 2. Those labels essentially assure separation of the key derivation function (KDF) input (which is depicted as $\mathcal{F}$). The initiator chooses a random bitstring n_I with bit-length greater or equal than 128 bits. The initiator generates ephemeral public-secret key pairs $(\mathsf{pk}_c, \mathsf{sk}_c)$, $(\mathsf{pk}_{pq}, \mathsf{sk}_{pq})$ via the key generation of the classic and post-quantum KEM, respectively. (If no public-secret key pair $(\mathsf{pk}_c, \mathsf{sk}_c)$ is generated, then the initiator sets pk_c and sk_c to empty strings.)

The initiator sends the public keys and the random values to the responder via a public channel.

The responder chooses a random bitstring n_R with bit-length k (with k greater or equal than 128 bits). The responder generates an encapsulation (c_{pq}, ss_{pq}) via the encapsulation algorithm of the post-quantum KEM using the public keys pk_{pq}. If pk_c is available at the responder, the

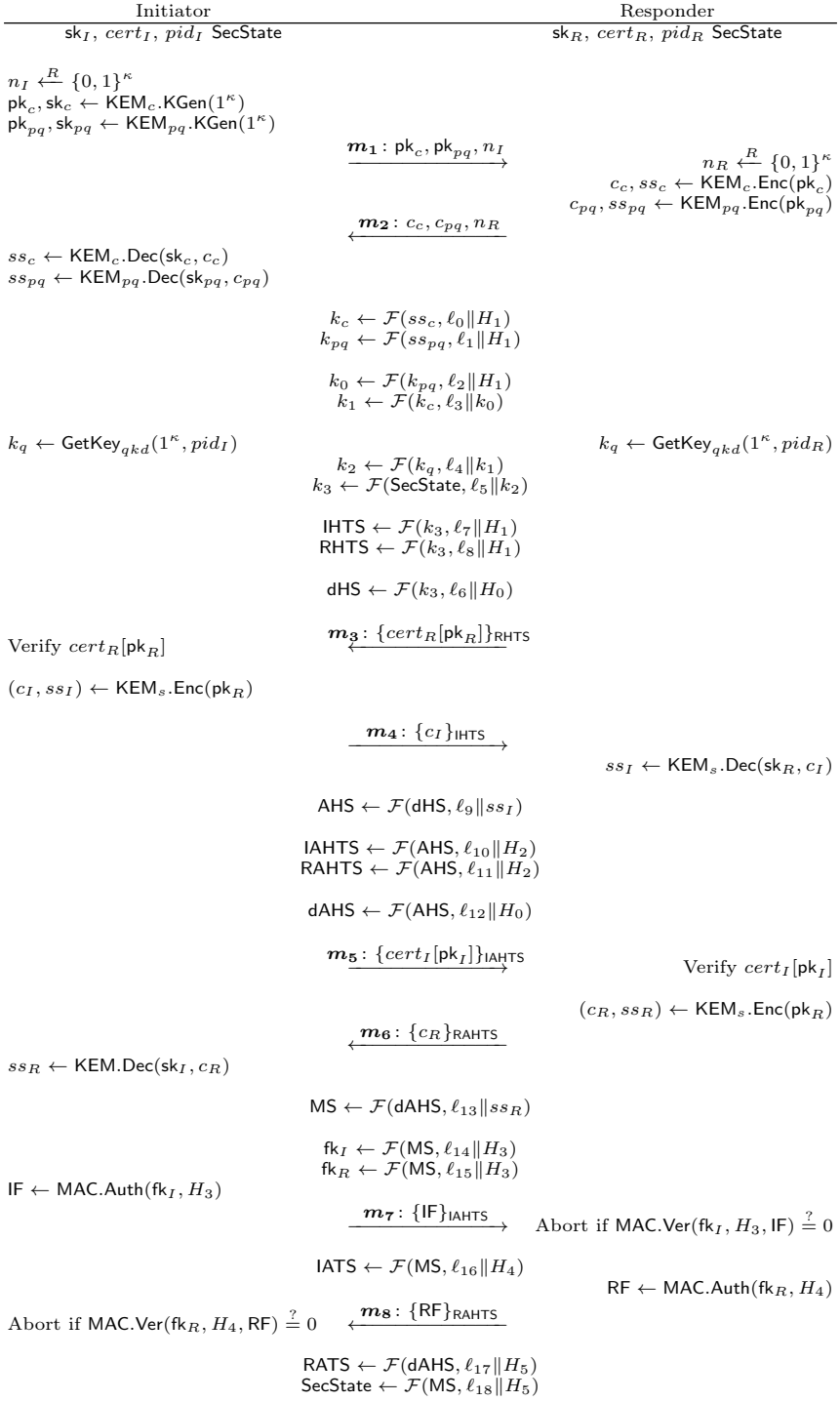

Fig. 1. One stage of the Muckle# protocol. We have a classical KEM KEM_c, a post-quantum KEM KEM_{pq}, a MAC MAC, a pseudorandom function $\mathcal{F}$, a post-quantum KEM KEM_s used for authentication, and a symmetric key k_q obtained from the $\mathtt{QKD}$ component (provided out-of-band via function GetKey_{qkd}). We assume that the certificates $cert_I$ and $cert_R$ contain the long-term public keys pk_I and pk_R, respectively. Messages $\boldsymbol{m_i}$: $\{m_{i,1}, \ldots\}_k$ denote that $m_{i,1}, \ldots$ is encrypted with an authenticated encryption scheme using the secret key k. The various contexts and labels are given in Tables 1 and 2.

responder generates an encapsulation (c_c, ss_c) via the encapsulation algorithm of the classical KEM using the public key pk_c. If pk_c is an empty string, then c_c and ss_c are set as empty strings.

The responder sends c_{pq}, c_c, and n_R to the initiator via a public channel. The initiator computes the decapsulation ss_{pq} and potentially ss_c using the decapsulation algorithm of the post-quantum KEM and the classical KEM using the secret key sk_{pq} and sk_c with the ciphertexts c_{pq} and c_c, respectively. If c_c is an empty string, then ss_c is set as an empty string.

Now, the initiator and responder invoke a series of KDF calls as given in Figure 1 where k_q is the symmetric QKD key. This essentially ensures a cryptographically sound way of "binding" the different keys and SecState together (which will argue about in the security proof).

The responder computes the encryption of $cert_R$ using the encryption algorithm of the AEAD with encryption key RHTS (and with associated data string "Message 3") and must send the resulting ciphertext m_3 to the initiator. The initiator computes the decryption of the received ciphertext m_3 using the decryption algorithm of the AEAD with decryption key RHTS and must verify the certificate $cert_R$ (where latter is out of scope of this work). Both parties have a value dHS.

Now, the authentication phase starts. The initiator computes the encapsulation ciphertext and key c_I, ss_I under the retrieved long-term public key pk_R of the receiver using the encapsulation algorithm of the post-quantum KEM and sends the c_I (AEAD-encrypted under IHTS) to the receiver.

The receiver computes the decapsulation key ss_I under the retrieved ciphertext c_I (AEAD-decrypted under IHTS) using the decapsulation algorithm of the post-quantum KEM. Both parties are able to retrieve the value dAHS via several calls to the KDF (see that also ss_I is given as input to one KDF call). The correctness of the post-quantum KEM guarantees that starting from the same value dHS, the initiator and receiver come to the same value dAHS *and* the responder is authenticated.

Next, the protocol does the same analogously for the initiator and, hence, both derive at the "master value" value MS *and* are mutually authenticated. What is missing is the key confirmation which is carried out on both sides via MAC key derived from MS. Finally, after successful mutual key confirmations, the secure state SecState gets updated via the KDF using the master value MS. The secure state will input to the next stage of the protocol (once triggered).

Under the assumption of the correctness of the KEM and MAC building blocks, correctness of Muckle# follows.

Binding, Intended Partners, and the QKD Link. Each party in a session of Muckle# has an input parameter indicating their intended session partner. Implicitly, when certificates are received, this is the value that the parties are checking against.

On the other hand, as discussed, we model the entire QKD link out-of-band, including its authentication. As such, in our call to the QKD link we need to explicitly specify the intended partner with whom we wish to establish a QKD key.

Notice that in the pre-shared key variant of TLS 1.3 [DHP20], parties are required to ensure, at the start of the protocol, that they are working from the same pre-shared key. This is accomplished by deriving a "binding key", that is used to compute a MAC of some pre-arranged label. We do not require this property if we are happy to insist that our long-term authentication secrets are not compromised before the session completes, since the authentication mechanism will detect a disagreement between the two parties on the key established by the QKD link. However, if we wish to allow everything but the QKD link to fail, we would require the derivation of a binding key to ensure that each local copy of the session is using secrets derived from the same key established by the QKD link. We refer the reader to the pre-shared key variant of TLS 1.3 for brief details how this would be done.

3.1 Security of Muckle#

The security proof is inspired by ideas in the realm of quantum-safe key exchanges. In particular we will use a security game in the tradition of the Bellare-Rogaway style key-indistinguishability games

[BR93], adapted to the hybrid setting in [DHP20]. Roughly speaking, we consider an adversary who controls *all* communication between honest parties, and has access to various oracles by which it can learn stage keys, ephemeral secrets, long-term secrets, and so on; the idea is that the adversary should not be able to do any better than simply acting passively as a wire, and transmitting the real information sent by the parties.

The HAKE framework was presciently set up to cover a wide variety of protocols in the hybrid setting. As such we do not have to adapt the framework to cater for our protocol – for example, the compromise of long-term KEM keys is the same as the compromise of long-term signature keys from the point of view of the relevant HAKE queries. Our security proof, however, also draws heavily from the techniques used in the security proof of KEMTLS [SSW20a]. Unlike our analysis, the security proof of KEMTLS treats that protocol as establishing several internal "stage keys," each of which can be revealed via queries available to the adversary. By contrast, we only deal with the indistinguishability of the final master secret; nevertheless, in the course of our security proof, we will show that the keys used internally for symmetric encryption, which could reasonably be described as "stage keys" in the model of KEMTLS, are indistinguishable from random.

The Muckle# cleanness predicate. As is standard in this area, we need to define a permissible set of queries which the adversary is allowed to issue. We do this by considering a stage "fresh" only if certain queries or combinations of queries have not been issued, and aborting if the tested stage is not fresh. This is both to cover trivial breaches – for example, a Reveal and a Test query being issued on the same stage – but also to define the scope of robustness of the protocol; that is, which secrecy breaches we can tolerate while maintaining a small advantage in the HAKE key-indistinguishability experiment.

The list of combinations of queries that prevent the freshness of a stage is called the "cleanness predicate;" both of [DHP20] and [BRS23] define bespoke cleanness predicates to cater to their particular authentication mechanisms. We define a cleanness predicate tailored for our context. In this direction, we note that an adversary $\mathcal{A}$ has access to all queries defined in the HAKE framework. As no pre-shared key exists in the (first-time version of the) Muckle# protocol, the query CorruptSK will return $\perp$ if called.

The predicate itself is, aside from the trivial controls on Reveal queries, divided into a passive and active case (as discussed, captured by the notion of matching sessions). The idea is that if the adversary is passive it suffices for the private ephemeral keys not to be leaked if the ind-cpa security of the KEMs holds; if the adversary is active, ephemeral KEMs do not offer any security by themselves, so we require that the long-term authentication secrets do not fail before the stage accepts. A Muckle# session π_i^s in stage t is *clean* if:

- Reveal(i, s, t) has not been issued
- Reveal(j, r, t) has not been issued for sessions π_j^r matching π_i^s at stage t.
- If π_i^s has a matching session π_j^r; at least one of the following is true:
  - If π_i^s has the initiator role in stage t, CompromiseQK(i, s, t) has not been issued before $\pi_i^s[t]$ accepts; if it has the responder role, CompromiseQK(j, r, t) has not been issued before before $\pi_j^r[t]$ accepts
  - If π_i^s has the initiator role in stage t, CompromiseSK(i, s, t) has not been issued; if it has the responder role, CompromiseSK(j, r, t) has not been issued
- If there is no (j, r, t) such that π_j^r is an origin session of π_i^s in stage t, at least one of the following holds:
  - CorruptQK(i) has not been issued before π_j^r accepts
  - CompromiseSK(i, s, t) has not been issued

Resumption stages and symmetric authentication. Muckle# keeps track of the internal variable SecState, which on the first stage of the session is initialized to some public value, and updated as a PRF evaluation of the master secret at the end of the stage. This new value is then fed into a subsequent stage of the protocol.

In subsequent stages of the session (say stage i), we can tolerate ephemeral and even long-term secrets failing, provided that we are willing to assume that CompromiseSS was not issued

in stage $i-1$ (arguing by a similar line to that in the proof). On the other hand, if we wish to add this scenario to the cleanness predicate, we might wish to define a version of Muckle# that uses more efficient symmetric authentication (or even does not inject further ephemeral secrets), analogously to the resumption sessions of TLS 1.3 [DFGS21]. Such a protocol, more or less, already exists; the original Muckle protocol [DHP20]. As such one can imagine a scenario where we define Muckle# as having its first stage consisting of the full protocol, and subsequent stages with PSK authentication. One would in this case have to be a little more careful with the cleanness predicate, and we leave this exercise to further work.

Theorem 1. *Let $\mathcal{F}$ be a dual PRF,* MAC *be an* $\mathsf{EUF\text{-}CMA}$ *secure MAC,* KEM_c *and* KEM_{pq} *be* ind-cpa *secure KEMs, and* KEM_s *be an* ind-cca *secure KEM. Then the Muckle# key exchange protocol is HAKE secure with the cleanness predicate* $\mathsf{clean}_{Muckle\#}$*. If the security of $\mathcal{F}$,* MAC*,* KEM_{pq} *and* KEM_s*, or of* `QKD` *holds, then so does the security of Muckle#.*

Proof. We begin by guessing the parameters defining the stage to be tested.

Game A0: This is the standard HAKE-experiment for an PPT adversary $\mathcal{A}$. For notational convenience we write

$$\mathsf{Adv}^{\mathsf{HAKE},\mathsf{clean}_{\mathrm{Muckle\#}},C_5}_{\mathcal{A},\mathrm{Muckle\#},n_p,n_s,n_t}(\kappa) =: \mathsf{Adv}^{A_0}_{\mathcal{A}}(\kappa).$$

Game A'0: In Game A'0, we guess the parameters (i,s,t) defining a stage to be tested, and the intended partner session in that stage. If $\mathsf{Test}(i',s',t')$ is issued with $(i,s,t) \neq (i',s',t')$ we abort the session. The advantage is

$$\mathsf{Adv}^{A_0}_{\mathcal{A}}(\kappa) \leq n_P^2 n_S n_T \mathsf{Adv}^{A'_0}_{\mathcal{A}}(\kappa)$$

Since a Test query issued on any other session to the one we guessed causes an abort, henceforth we can talk about *the* tested session (at the cost of incurring some polynomial factor in the bound).

We now split into two cases: the case that the tested session does not have an origin session in stage t, and the case that it does. We start with the former case.

Case 1: no origin session. Suppose that a session π^s_i does not have an origin session in stage t. Intuitively, the ephemeral KEMs provide no security here, since our opponent is active and can encapsulate their own choice of secret; the strategy of the proof is to start by replacing the secret encapsulated by the long term KEM, and then proceed by a series of game hops to an unwinnable game. For this replacement to be sound, we require that the tested session's intended partner's long-term secret, or the replacement, is trivially detected. If the long-term secret has been compromised, it suffices for the `QKD` link to survive.

Subcase 1a: No $\mathsf{CorruptQK}(i,s,t)$ *has been issued.* The arguments are slightly different, depending on whether the tested session was an initiator or a responder. In the interest of handling the more complex case first, we may for now suppose that the tested session is a responder. In this subcase, replacement of the initiator's static secret ss_I is non-problematic.

Sub-subcase 1a': π^s_i is a responder in stage t.

Game A'0: This is the game $A'0$ described above, with advantage $\mathsf{Adv}^{A'0}_{\mathcal{A}}(\kappa)$.

Game A'1: In Game $A1$ we replace ss_I with a value $\widetilde{ss_I}$ sampled uniformly at random from the output space of $\mathsf{KEM}_s.\mathsf{Enc}()$. Any subsequent value computed as a function of ss_I is computed instead as a function of $\widetilde{ss_I}$. Consider our adversary $\mathcal{A}$ given the data defining either game $A'0$ or game $A1$ (notice the syntax of both games is the same); we will relate $\mathcal{A}$'s advantage in each of these games by an adversary tackling the ind-cca security game. Let $\mathcal{A}1$ be such an adversary against the ind-cca security of KEM_s. Suppose $\mathcal{A}1$ receives challenge $(\mathsf{pk}^*, c^*, ss^*)$ in the ind-cca game. Since the session is assumed to be clean, $\mathcal{A}$ will not issue a $\mathsf{CompromiseQK}$ query, so we may safely replace the long-term public key of the corrupted partner with pk^*. We can also replace the initiator's encapsulation (c_I, ss_I) with the challenge (c^*, ss^*). Notice,

however, that because the tested responder session does not have an origin session, we are not guaranteed that c^* is delivered. If this happens, let c'_I received by the tested session. The ind-cca adversary, in this case, queries its decapsulation oracle on c'_I and uses the response as ss^*. If ss^* was the real secret that ct^* was encapsulated against, the view of $\mathcal{A}$ is exactly as though it is playing game $A1$; otherwise it is exactly as though it is playing game $A2$. If we use the output of $\mathcal{A}$ as the guess for $\mathcal{A}1$ it follows that

$$\mathsf{Adv}^{A'_0}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{A_1}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{ind\text{-}cca}}_{\mathcal{A}1,\mathsf{KEM}_s}(\kappa)$$

Game A'2: In Game $A2$ we replace AHS with a value $\widetilde{\mathsf{AHS}}$ chosen uniformly at random from the output space of $\mathcal{F}$. Again, any values computed as a function of AHS are instead computed as a function of $\widetilde{\mathsf{AHS}}$. We employ a similar argument to the above. Since the adversary is active and the $\mathtt{QKD}$ link has not been assumed to have survived, we may assume access to dHS. An adversary $\mathcal{A}2$ trying to win the $\mathsf{dual}-\mathsf{PRF}$ security game can query their PRF oracle on dHS and run $\mathcal{A}$ on Game $A1$ with AHS replaced with the oracle response. If the oracle was indeed a pseudorandom function we have simulated Game A1 to $\mathcal{A}$; otherwise we have simulated Game A2 to $\mathcal{A}$, and so

$$\mathsf{Adv}^{A1}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{A2}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{A}2,\mathcal{F}}(\kappa)$$

Game A'3: In Game $A3$, IAHTS is replaced with a value sampled uniformly at random, $\widetilde{\mathsf{IAHTS}}$. Any value derived from IAHTS is instead computed as a function of $\widetilde{\mathsf{IAHTS}}$. An adversary $\mathcal{A}3$ trying to win the $\mathsf{dual}-\mathsf{PRF}$ security game can query its oracle on $\ell_9 \| H_2$ and run $\mathcal{A}$ on Game $A2$ with IAHTS replaced with the oracle response. Since in game $A2$, IAHTS is computed from $\widetilde{AHS}$, which is a uniformly random value not known to the adversary, if the oracle response was indeed from a pseudorandom function we have exactly simulated Game $A3$ to $\mathcal{A}$; otherwise we have simulated $A3$. It follows that

$$\mathsf{Adv}^{A2}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{A3}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{A}3,\mathcal{F}}(\kappa)$$

Games A'4, A'5: We repeat the argument above for the values RAHTS and dAHS. With $\mathcal{A}4, \mathcal{A}5$ PRF adversaries defined similarly to the above one has

$$\mathsf{Adv}^{A3}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{A4}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{A}4,\mathcal{F}}(\kappa)$$

$$\mathsf{Adv}^{A4}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{A5}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{A}5,\mathcal{F}}(\kappa)$$

Indeed, without loss of generality we may suppose that $\mathcal{A}4$ and $\mathcal{A}5$ are the same adversary (since by assumption no PPT adversary has better than trivial advantage in the $\mathsf{dual}-\mathsf{PRF}$ game). This observation yields

$$\mathsf{Adv}^{A3}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{A5}_{\mathcal{A}}(\kappa) + 2\mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{A}4,\mathcal{F}}(\kappa)$$

We will implicitly call upon this observation several times.

Game A'6: In Game $A6$ we replace MS with $\widetilde{\mathsf{MS}}$, a value sampled uniformly at random from the output space of $\mathcal{F}$. Any values derived from MS are instead derived from $\widetilde{MS}$. The tested session is an initiator session and the adversary is active, so we may assume access to ss_R. A PRF adversary $\mathcal{A}6$ can therefore query its PRF oracle on $\ell_{13} \| ss_R$ and run $\mathcal{A}$ on game $A5$ with MS replaced by the oracle response. Since in Game $A5$, dAHS is a uniformly random value not known to the adversary, if the oracle output was from a pseudorandom function we have simulated Game $A5$ to $\mathcal{A}$. Otherwise, we have simulated $A6$, yielding

$$\mathsf{Adv}^{A5}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{A6}_{\mathcal{A}}(\kappa) + \mathcal{A}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{A}6,\mathcal{F}}(\kappa)$$

Game A'7-A'11: Since MS is now replaced with a uniformly random value, we may sequentially replace $\mathsf{fk}_I, \mathsf{fk}_R, \mathsf{IATS}, \mathsf{RATS}, \mathsf{SecState}$ with uniformly random values. As usual, any derivatives of these values are instead computed as derivatives of the replacement random values. By

considering PRF adversaries $\mathcal{A}7-11$ proceeding according to the strategy outlined above (and recalling our observation in Games A4 and A5), we conclude that

$$\mathsf{Adv}^{A6}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{A11}_{\mathcal{A}}(\kappa) + 5\mathsf{Adv}^{\mathsf{dual-PRF}}_{\mathcal{A}7,\mathcal{F}}(\kappa)$$

Identical-until-bad We now define the event 'bad' as that whereby the initiator accepts a tag RF. Let B_0 denote the event that the 'bad' event occurs in Game A11.

Game A'12: Game A12 is exactly the same as Game A11, except we abort if the bad event occurs. Since A11 and A12 are identical-until-bad, by [BR04, Lemma 2] we have

$$|\mathsf{Adv}^{A11}_{\mathcal{A}}(\kappa) - \mathsf{Adv}^{A12}_{\mathcal{A}}(\kappa)| < \Pr(B_0)$$

Notice that in Game A12, because the adversary can only win the game if the session accepts, but we abort whenever a session accepts because of the bad event, trivially we have $\mathsf{Adv}^{A12}_{\mathcal{A}}(\kappa) = 0$. It remains to bound $\Pr(B_0)$.

Bounding $\Pr(B_0)$**:** Recall we are in the subcase where there is at least one $\boldsymbol{m_i}$ that is not faithfully delivered by the adversary. Without loss of generality we may suppose $\boldsymbol{m_7}$ was not delivered faithfully - because in Game A11, all the secrets have been succesfully replaced by uniformly random values. If $\boldsymbol{m_7}$ is faithfully delivered then the adversary has acted passively in a game in which all the secrets are uniformly random values, and so its advantage is 0.
Consider an adversary $\mathcal{A}8$ playing the $\mathsf{EUF\text{-}CMA}$ security game with respect to a symmetric key fk_I, that runs $\mathcal{A}$ playing Game A11 as a subroutine. We answer its oracle queries under fk_I. Recalling that $\mathcal{A}$ controls all traffic; when it produces IF, we submit (H_3, IF) as our forgery. We answer $\mathcal{A}8's$ oracle queries with the key fk_I. Since fk_I has been replaced with a uniform random value, the simulation is sound; and by the above we may assume that no honest origin session delivder $\boldsymbol{m_7}$. This is therefore a valid forgery, and so the probability that B_0 occurs is bounded by the advantage of $\mathcal{A}8$ in the $\mathsf{EUF\text{-}CMA}$ security game; that is,

$$\Pr(B_0) \leq \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathcal{A}8}(\kappa)$$

Descending the chain of inequalities, we conclude that there are PPT adversaries $\mathcal{A}1, \mathcal{A}2$ and $\mathcal{A}3$ such that

$$\mathsf{Adv}^{A0}_{\mathcal{A}}(\kappa) \leq \left(\mathsf{Adv}^{\mathsf{ind\text{-}cca}}_{\mathcal{A}1,\mathsf{KEM}_s}(\kappa) + 7\mathsf{Adv}^{\mathsf{dual-PRF}}_{\mathcal{A}2,\mathcal{F}}(\kappa) + \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathcal{A}3,\mathsf{MAC}}(\kappa)\right)$$

Sub-subcase 1a": π^s_i is an initiator in stage t. Suppose that the tested session is an initiator. We employ the same general argument; but now we note that the replacement of the initiator's secret ss_I is no longer sound. Instead, notice that we can now replace ss_R with a value sampled uniformly at random and again relate the advantage of the two resulting games by the advantage of and $\mathsf{ind\text{-}cca}$ adversary. In a version of the game where ss_R is a uniformly random value not known to the adversary we may regard $\mathcal{F}$ as pseudorandom in its first argument, so we can proceed by the same argument as in the initiator case and invoke the $\mathsf{dual-PRF}$ security of $\mathcal{F}$, and replace MS with a value sampled uniformly at random. The proof is then the same as in the initiator case, and it follows that there are PPT adversaries $\mathcal{A}1, \mathcal{A}2, \mathcal{A}3$ such that

$$\mathsf{Adv}^{A'0}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{\mathsf{ind\text{-}cca}}_{\mathcal{A}1,\mathsf{KEM}_s}(\kappa) + 6\mathsf{Adv}^{\mathsf{dual-PRF}}_{\mathcal{A}2}(\kappa) + \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathcal{A}3,\mathsf{MAC}}(\kappa)$$

Subcase 1b: no $\mathsf{CompromiseSK}(i,s,t)$ *has been issued.* Having handled the subcase in which the long-term authentication secrets were not compromised, we may now consider the subcase in which no $\mathsf{CompromiseSK}(i,s,t)$ query was issued. In this case the proof is the same for a tested initiator and tested responder session. The idea is that if the $\mathtt{QKD}$ link was not compromised we can consider k_q to be a uniformly random value, after which we may proceed by similar arguments to the above.

Game A'0: This is the same Game A'0 defined above.

Game B1: This game is the same as the previous one, except we replace the chaining key k_2 with a value $\widetilde{k_2}$ sampled uniformly at random. Any values derived from k_2 are instead derived from $\widetilde{k_2}$. As discussed above, we regard k_q as a value distributed uniformly at random, because the QKD link is assumed to have survived. Since we are in the active case a $\mathsf{dual}-\mathsf{PRF}$ adversary $\mathcal{B}1$ can be assumed to have access to k_1; this adversary can therefore query its PRF oracle on $\ell_4\|k_1$, and run $\mathcal{A}$ on Game 0 with k_2 replaced with the oracle output. If the oracle was a PRF then we have exactly simulated Game 0; otherwise we have exactly simulated $B1$, yielding

$$\mathsf{Adv}^{A'_0}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{B_1}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{B}1,\mathcal{F}}(\kappa)$$

Game B2: In this game we replace the chaining key k_3 with the a value sampled uniformly at random, say $\tilde{k_3}$. Any values derived from k_3 are instead derived from $\tilde{k_3}$. If SecState is known to the adversary (either because it is the first stage of the session, or because a CompromiseSS query was issued on the prior stage), a $\mathsf{dual}-\mathsf{PRF}$ adversary $\mathcal{B}2$ can query its oracle on SecState and run $\mathcal{A}$ on Game $B2$ with k_3 replaced by the oracle output. Since $\mathcal{F}$ is also a PRF on its first argument in this case, if the oracle output was from a PRF we have exactly simulated Game $B1$; otherwise we have exactly simulated $B2$, whence

$$\mathsf{Adv}^{B_1}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{B_2}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{B}2,\mathcal{F}}(\kappa)$$

Notice that if SecState was not known, the $\mathsf{dual}-\mathsf{PRF}$ adversary can instead query its oracle on a value chosen uniformly at random, and the same argument applies.

Games B3-5: IHTS, RHTS and dHS are now computed as PRF evaluations on the uniformly random value $\tilde{k_3}$. By considering PRF adversaries $\mathcal{B}3, \mathcal{B}4$ and $\mathcal{B}5$ running $\mathcal{A}$ on games $B3, B4$ and $B5$ respectively, and arguing as above, one has

$$\mathsf{Adv}^{B2}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{B5}_{\mathcal{A}}(\kappa) + 3\mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{B}3}(\kappa)$$

Game B6: In this game we replace AHS with a uniform random value, $\widetilde{AHS}$. Any values derived from AHS are instead derived from $\widetilde{AHS}$. Note that we are in the active adversary scenario so we may assume knowledge of ss_I. A PRF adversary $\mathcal{B}6$ can therefore query its oracle on $\ell_9\|ss_I$ and run $\mathcal{A}$ on Game $B5$ with AHS replaced by the oracle response. Since in this game dHS has been replaced with a uniform random value not known to the adversary, if the oracle was a PRF we have exactly simulated Game $B5$ to $\mathcal{A}$; otherwise we have simulated Game $B6$, giving

$$\mathsf{Adv}^{B5}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{B6}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{B}6,\mathcal{F}}(\kappa)$$

Games B7-9: In these games AHS has been replaced by a uniformly random value. Following the usual argument, we define games $B7 - B9$ by sequentially replacing IAHTS, RAHTS and dAHS, and consider PRF adversaries $\mathcal{B}7 - 9$ running $\mathcal{A}$ on these games. We obtain

$$\mathsf{Adv}^{B6}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{B9}_{\mathcal{A}}(\kappa) + 3\mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{B}7,\mathcal{F}}(\kappa)$$

Game B10: In this game we replace MS with a uniform random value, say $\widetilde{MS}$. Any values derived from MS are instead derived from $\widetilde{MS}$. A PRF adversary $\mathcal{B}8$ can query its PRF oracle on $\ell_1 3\|ss_R$ (where we may assume knowledge of ss_R since we are we did not require that the long-term authentication mechanism survives) and run $\mathcal{A}$ on Game $B9$ with MS replaced by the oracle response. Since dAHS has already been replaced by a uniformly random value not known to the adversary, if the oracle was a PRF we have exactly simulated Game $B9$ to $\mathcal{A}$; otherwise, we have simulated Game $B10$. It follows that

$$\mathsf{Adv}^{B9}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{B10}_{\mathcal{A}}(\kappa) + \mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{B}8,\mathcal{F}}(\kappa)$$

We are now in exactly the same scenario as Game $A7$ and may proceed according to the same argument. We conclude that there are PPT adversaries $\mathcal{B}1, \mathcal{B}2$

$$\mathsf{Adv}^{A_0}_{\mathcal{A}}(\kappa) \leq 15\mathsf{Adv}^{\mathsf{dual}-\mathsf{PRF}}_{\mathcal{B}1,\mathcal{F}}(\kappa) + \mathsf{Adv}^{\mathsf{EUF\text{-}CMA}}_{\mathcal{B}2,\mathsf{MAC}}(\kappa)$$

Case 2: origin session exists. We can now move on to the case where the tested stage π_i^s has a matching session π_j^r in stage t. As we have discussed, this can be thought of as the "passive" case. Here we only need one of the ephemeral post-quantum secret, the long-term post-quantum secret, the per-stage $\mathsf{SecState}$ (provided $t > 1$), or the out-of-band $\mathtt{QKD}$ link to survive.

Again, all the game-hops here happen from Game A'0 that we defined before, allowing us to argue about a single session being tested, at the cost of a polynomial factor in the final bound.

In all of these cases we use exactly the same principles as above to show that if a secret is not compromised, we can set up a PRF game such that the advantage in this PRF game bounds the difference in advantage between the original game and a version of the original game whereby the secret is replaced with a uniformly random value. Notice also that in this matching case we do not have to worry about MAC forgeries. As such we eschew the details of the proof in these cases and present the following conclusions in terms of advantage.

If no $\mathsf{CompromiseSK}(i, s, t)$ or $\mathsf{CompromiseSK}(j, r, t)$ has been issued, we can (regardless of the tested stage's role) replace ss_{pq} with a value chosen uniformly at random, $\widetilde{ss_{pq}}$. Since the adversary is passive, this is a sound replacement. Similarly to what we saw in the active case, we can use the advantage in a specially constructed $\mathsf{ind\text{-}cpa}$ game to bound the difference between the advantage of the original key-indistinguishability game, and that of a variant of this game, in which the ephemeral post-quantum secret is uniformly random. From there we can sequentially replace secrets by relying on the $\mathsf{dual-PRF}$ security of $\mathcal{F}$. Moreover, since we are in the passive case, we do not have to worry about MAC forgeries, so we will not need to invoke the 'identical-until-bad' type argument of the previous cases. It follows that in this case, for a PPT adversary $\mathcal{A}$ we have PPT adversaries $\mathcal{C}1, \mathcal{C}2$ such that

$$\mathsf{Adv}^{A_0}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{\mathsf{ind\text{-}cpa}}_{\mathcal{C}1,\mathsf{KEM}_{pq}}(\kappa) + 18\mathsf{Adv}^{\mathsf{dual-PRF}}_{\mathcal{C}2,\mathcal{F}}(\kappa)$$

If no $\mathsf{CompromiseSK}(i, s, t)$ or $\mathsf{CompromiseSK}(j, r, t)$ have been issued the proof goes through almost exactly the same as in the previous case, except we no longer have to bound the probability of a MAC forgery (since we are in the passive scenario). In this case we proceed entirely by relying on the $\mathsf{dual-PRF}$ security of $\mathcal{F}$. In particular, for a PPT adversary $\mathcal{A}$ we have a PPT adversary $\mathcal{D}$ such that

$$\mathsf{Adv}^{A_0}_{\mathcal{A}}(\kappa) \leq 15\mathsf{Adv}^{\mathsf{dual-PRF}}_{\mathcal{D},\mathcal{F}}(\kappa)$$

Finally, we notice that we can allow everything apart from the long-term authentication secrets to fail, because KEM authentication introduces new secrecy into the key schedule. (Compare this with signature authentication, in which a passively-observed transcript where the signature keys survive but ephemeral keys fail yields full key recovery). If the tested stage is a responder session, for a PPT adversary $\mathcal{A}$ there are PPT adversaries $\mathcal{E}1, \mathcal{E}2$ such that

$$\mathsf{Adv}^{A_0}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{\mathsf{ind\text{-}cpa}}_{\mathcal{E}1,\mathsf{KEM}_s}(\kappa) + 9\mathsf{Adv}^{\mathsf{dual-PRF}}_{\mathcal{E}2,\mathcal{F}}(\kappa)$$

If the tested stage was a responder the bound is

$$\mathsf{Adv}^{A_0}_{\mathcal{A}}(\kappa) \leq \mathsf{Adv}^{\mathsf{ind\text{-}cpa}}_{\mathcal{E}1,\mathsf{KEM}_s}(\kappa) + 5\mathsf{Adv}^{\mathsf{dual-PRF}}_{\mathcal{E}2,\mathcal{F}}(\kappa)$$

Notice that in comparison with the active case, we now only need the $\mathsf{ind\text{-}cpa}$ security of KEM_s.

Remark 2. In contrast to KEMTLS [SSW20a], we do not require the ephemeral KEMs to have $\mathsf{ind\text{-}1cca}$ security (and instead only require $\mathsf{ind\text{-}cpa}$ security). This is because the proof of KEMTLS and our proof split into two cases slightly differently. In the proof of KEMTLS, there is a case whereby an honest matching session exists, and a case in which an honest matching session does not exist, but in both scenarios the adversary is allowed to behave actively. There is no guarantee that the adversary faithfully delivers the encapsulation under the ephemeral KEM, so for the security proof to go through, we may require a decapsulation query in order to replace the ephemeral secret in a valid fashion. In contrast, our proof divides into a case where the session we partner with has a corresponding session with a matching transcript, which implies that the adversary has acted

passively. As such, we know that the encapsulation of the ephemeral secret has been delivered faithfully, and so we do not need to rely on a decapsulation query. In the case where there is no session with a matching transcript, the adversary must have behaved actively – but in this case, the ephemeral KEMs do not offer any security anyway, since an active adversary can safely encapsulate their own secret against the initiator's public key (or send their own ephemeral public keys to a responder), and we rely on the authentication mechanism. In summary, in either of our cases, there is no need to upgrade the ind-cpa security of the ephemeral KEM to ind-1cca security. Note that this is in line with the results from Huguenin-Dumittan and Vaudenay [HV22] where the authors show that CPA-secure ephemeral KEMs are essentially sufficient for TLS 1.3. Our work shows that such guarantees also hold for Muckle# in the HAKE framework.

4 Implementation and Evaluation

We implemented a prototype of the Muckle# protocol in Python using bindings[13] of `liboqs` [SM16] for the support of post-quantum signature schemes and KEMs as well as the `cryptography`[14] module for all classically-secure schemes. The initiator (and the responder) were executed on a notebook running Windows 10 with an Intel i5 2.60GHz CPU and 8 GB of RAM reflecting a client (and server) view.

About the implemented architecture. The architecture of an application integrating the Muckle# protocol is depicted in Figure 2. The protocol is implemented in the application layer of a `QKD` network. The quantum key material is fetched by the key managements services (KMSs) [JLRT23,MBO+23,MNR+20] from the `QKD` devices and is then provided via the ETSI GS QKD 014 [Eur19] interface to the application layer. With this interface, the initiator-side of the application obtains a key from the KMS together with an associated ID. For the responder to request the same key, the initiator needs to communicate the key ID to the responder which then requests the key by providing the key ID. Hence, this ID needs to be included in the initial message of the key exchange but no other information needs to be transmitted for the initiator and responder to use the ETSI GS QKD 014 interfaces.

To go into more detail regarding the additional communication cost, we want to note that our protocol requires one more message compared to TLS 1.3 from the initiator (i.e., the client in the TLS setting) to the responder (i.e., the server). The initiator/client is the first party to be able to send encrypted data instead of the responder/server. The server being able to send the first encrypted message is only useful for certain protocols such as SMTP, but not for HTTPS where the client sends the message. For client-initiated protocols, the additional message has no negative impact. The concrete trade-offs depend, of course, on the concrete choice of primitives.

Further requirements and assumptions. We evaluated the implementation of Muckle# with a simulated link with a single post-quantum secure certificate authority. We used simulators instead of actual `QKD` devices to ensure that the limited key rate does not influence the practical results (a `QKD` key has 32 bytes). The benchmark was performed using mutual authentication and hence both parties authenticated themselves using certificates. The certificates contained both post-quantum and classical long-term public keys. They were also authenticated in a two-layer certificate hierarchy, i.e., with one root CA and one intermediate CA. The CA signed the certificates using both a post-quantum signature scheme (ML-DSA-87) and a classical signature scheme, namely EdDSA [BDL+11].

Concrete analysis. The performance is evaluated based on two critical metrics: bandwidth and CPU cycles. The following analysis compares these metrics for different schemes as depicted in Table 3.

[13] `https://github.com/open-quantum-safe/liboqs-python`
[14] `https://pypi.org/project/cryptography/`

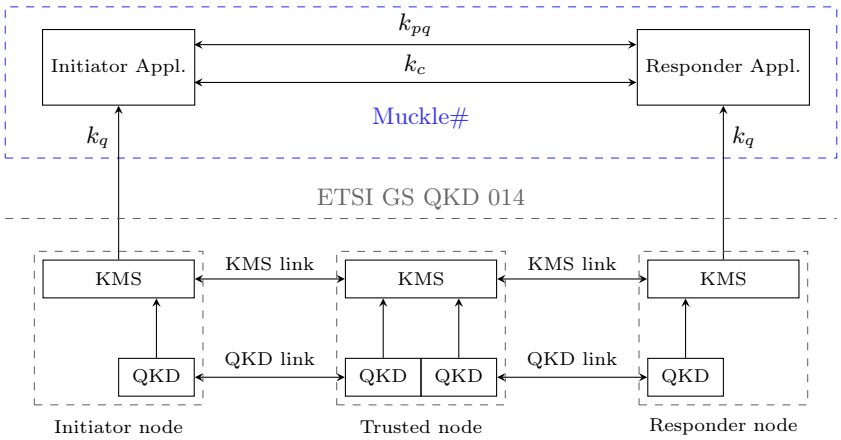

Fig. 2. Architecture of the Muckle# implementation. k_q represents the `QKD` key, k_{pq} the key obtained from a post-quantum secure KEM, and k_c the key obtained from a classically secure KEM. KMS represents the key management system. Note that the protocol supports any number of trusted nodes in between. The architecture is adapted from Muckle+ from [BRS23].

Table 3. Bandwidth and CPU usage in kilobytes and giga cycles, respectively, for Muckle# over 5 runs. We use ML-DSA-87 for the post-quantum signature scheme of the long-term KEM keys. (Unfortunately, we were not able to measure cycles for FrodoKEM-976-SHAKE due to an error in a software library.)

Ephemeral Keys	Long-Term Keys	Cycles (G)	Data (KB)
ML-KEM-512, ECDH (X25519), QKD	ML-KEM-512	1.6	29.2
ML-KEM-768, ECDH (X25519), QKD	ML-KEM-768	1.7	31.3
ML-KEM-1024, ECDH (X25519), QKD	ML-KEM-1024	1.7	33.9
HQC-128, ECDH (X25519), QKD	HQC-128	2.9	44.5
HQC-256, ECDH (X25519), QKD	HQC-256	6.0	89.4
FrodoKEM-640-SHAKE, ECDH (X25519), QKD	FrodoKEM-640-SHAKE	1.8	82.5
FrodoKEM-976-SHAKE, ECDH (X25519), QKD	FrodoKEM-976-SHAKE	-	118.6
FrodoKEM-1344-SHAKE, ECDH (X25519), QKD	FrodoKEM-1344-SHAKE	2.4	153.9

The highest bandwidth is observed using FrodoKEM-1344-SHAKE for long-term keys at 153.9 KB, followed by FrodoKEM-976-SHAKE at 118.6 KB. ML-KEM-512 and ML-KEM-768 exhibit the lowest bandwidth at 29.2 KB and 31.3 KB, respectively. The schemes HQC-128 and HQC-256 have moderate bandwidth values of 44.5 KB and 89.4 KB, respectively.

The computational demand in terms of CPU cycles is as follows over 5 runs. HQC-256 for long-term keys requires the highest CPU cycles at 6.0 G cycles. HQC-128 and FrodoKEM-1344-SHAKE have moderate values of 2.9 G cycles and 2.4 G cycles, respectively. ML-KEM-512, ML-KEM-768 and ML-KEM-1024 are the most efficient with CPU cycles of 1.6, 1.7 and 1.7 G cycles, respectively, while FrodKEM-640-SHAKE has 1.8 G cycles.

Conclusion. Using Schemes such as ML-KEM-768 and ML-KEM-1024 in Muckle# as long-term keys for authentication are efficient in both bandwidth and CPU cycles while also guaranteeing a fairly suitable level of security. FrodoKEM-1344-SHAKE offers higher security but at the cost of increased bandwidth and computational requirements. Nevertheless, the choice of the scheme depends on the specific requirements of bandwidth efficiency versus computational performance in the specific post-quantum cryptographic applications.

Remark 3. Generally note, though, the major bottleneck is the key rate of the underlying QKD network. With key rates in the range of a few kilobits per second,[15] the cost of all involved cryptographic primitives and the cost of the transmission is insignificant. Nevertheless, since QKD is modelled in a black-box way in this work, one can expect the efficiency gains of the post-quantum primitives to be increasingly germane as the relevant technology improves.

Moreover, note that Muckle# can also benefit from the pre-distributed certificates [SSW21]. As the certificates are a major contributor to the size of the handshake messages,[16] pre-distribution of the certificates has a significant impact on the communication overhead. Considering the bandwidth cost measured during a single protocol run (c.f. Table 3), the certificates are the single largest contributor to the required bandwidth. Considering current efforts to standardize Encrypted Client Hello for TLS [ROSW24], out-of-band communication of certificates will become a practical possibility for TLS and other protocols to save communication overhead. This proposal foresees the distribution of certificates via DNS records which will cause other challenges to be tackled with respect to post-quantum signatures, e.g., to the limited sizes of DNS records [RJ23].

5 Conclusion and Outlook

We propose a novel variant of hybrid authenticated key exchange protocols, in which authentication solely relies on the availability of post-quantum KEMs, instead of digital signatures or pre-shared keys; and prove its security in the HAKE framework. In terms of future direction, we echo the sentiment in the conclusion of [DHP20]; that is, it would be interesting to integrate the physical models used to prove QKD into our framework, particularly with respect to the limitations set out in Section 1.1.

Acknowledgements. This work received funding from the European Union's Horizon Europe research and innovation programme under agreement number 101114043 ("QSNP") and from the Digital Europe Program under grant agreement number 101091642 ("QCI-CAT"). Authors CB and LP conducted this work with the support of ONR Grant 62909-24-1-2002. LP also thanks Google for supporting academic post-quantum research. Authors CB, CS, and LP acknowledge support of the Institut Henri Poincaré (UAR 839 CNRS-Sorbonne Université), and LabEx CARMIN (ANR-10-LABX-59-01).

References

BDL+11. Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. In Bart Preneel and Tsuyoshi Takagi, editors, *CHES 2011*, volume 6917 of *LNCS*, pages 124–142. Springer, Berlin, Heidelberg, September / October 2011. doi:10.1007/978-3-642-23951-9_9.

BL15. Mihir Bellare and Anna Lysyanskaya. Symmetric and dual PRFs from standard assumptions: A generic validation of an HMAC assumption. Cryptology ePrint Archive, Report 2015/1198, 2015. URL: https://eprint.iacr.org/2015/1198.

BM99. Mihir Bellare and Sara K. Miner. A forward-secure digital signature scheme. In Michael J. Wiener, editor, *CRYPTO'99*, volume 1666 of *LNCS*, pages 431–448. Springer, Berlin, Heidelberg, August 1999. doi:10.1007/3-540-48405-1_28.

BMO17. Raphaël Bost, Brice Minaud, and Olga Ohrimenko. Forward and backward private searchable encryption from constrained cryptographic primitives. In Bhavani M. Thuraisingham, David Evans, Tal Malkin, and Dongyan Xu, editors, *ACM CCS 2017*, pages 1465–1482. ACM Press, October / November 2017. doi:10.1145/3133956.3133980.

BR93. Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In *Annual international cryptology conference*, pages 232–249. Springer, 1993.

[15] For example see https://www.thinkquantum.com/quky/ for practical key rates of a device supplier and [DEG+23].

[16] A single ML-DSA-87 signature is 4627 bytes large, so the two signatures required for a setting with a root CA and an intermediate CA account for almost 9 KB.

BR95. Mihir Bellare and Phillip Rogaway. Provably secure session key distribution: The three party case. In *27th ACM STOC*, pages 57–66. ACM Press, May / June 1995. doi:10.1145/225058.225084.

BR04. Mihir Bellare and Phillip Rogaway. Code-based game-playing proofs and the security of triple encryption. *Cryptology ePrint Archive*, 2004.

BRS23. Sonja Bruckner, Sebastian Ramacher, and Christoph Striecks. Muckle+: End-to-end hybrid authenticated key exchanges. In Thomas Johansson and Daniel Smith-Tone, editors, *Post-Quantum Cryptography - 14th International Workshop, PQCrypto 2023*, pages 601–633. Springer, Cham, August 2023. doi:10.1007/978-3-031-40003-2_22.

BVB+24. Max Brauer, Rafael J. Vicente, Jaime S. Buruaga, Rubén B. Méndez, Ralf-Peter Braun, Marc Geitz, Piotr Rydlichowski, Hans H. Brunner, Chi-Hang Fred Fung, Momtchil Peev, Antonio Pastor, Diego R. López, Vicente Martín, and Juan Pedro Brito. Linking QKD testbeds across europe. *Entropy*, 26(2):123, 2024.

CD23. Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 423–447. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_15.

CF12. Cas J. F. Cremers and Michele Feltz. Beyond eCK: Perfect forward secrecy under actor compromise and ephemeral-key reveal. In Sara Foresti, Moti Yung, and Fabio Martinelli, editors, *ESORICS 2012*, volume 7459 of *LNCS*, pages 734–751. Springer, Berlin, Heidelberg, September 2012. doi:10.1007/978-3-642-33167-1_42.

CHK03. Ran Canetti, Shai Halevi, and Jonathan Katz. A forward-secure public-key encryption scheme. In Eli Biham, editor, *EUROCRYPT 2003*, volume 2656 of *LNCS*, pages 255–271. Springer, Berlin, Heidelberg, May 2003. doi:10.1007/3-540-39200-9_16.

CRSS20. Valerio Cini, Sebastian Ramacher, Daniel Slamanig, and Christoph Striecks. CCA-secure (puncturable) KEMs from encryption with non-negligible decryption errors. In Shiho Moriai and Huaxiong Wang, editors, *ASIACRYPT 2020, Part I*, volume 12491 of *LNCS*, pages 159–190. Springer, Cham, December 2020. doi:10.1007/978-3-030-64837-4_6.

DCM20. Emma Dauterman, Henry Corrigan-Gibbs, and David Mazières. Safetypin: Encrypted backups with human-memorable secrets. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*, pages 1121–1138. USENIX Association, 2020.

DDG+20. Fynn Dallmeier, Jan P. Drees, Kai Gellert, Tobias Handirk, Tibor Jager, Jonas Klauke, Simon Nachtigall, Timo Renzelmann, and Rudi Wolf. Forward-secure 0-RTT goes live: Implementation and performance analysis in QUIC. In Stephan Krenn, Haya Shulman, and Serge Vaudenay, editors, *CANS 20*, volume 12579 of *LNCS*, pages 211–231. Springer, Cham, December 2020. doi:10.1007/978-3-030-65411-5_11.

DEG+23. Christoph Döberl, Wolfgang Eibner, Simon Gärtner, Manuela Kos, Florian Kutschera, and Sebastian Ramacher. Quantum-resistant end-to-end secure messaging and email communication. In *ARES*, pages 125:1–125:8. ACM, 2023.

DFGS21. Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol. *Journal of Cryptology*, 34(4):37, October 2021. doi:10.1007/s00145-021-09384-1.

DGJ+21. David Derler, Kai Gellert, Tibor Jager, Daniel Slamanig, and Christoph Striecks. Bloom filter encryption and applications to efficient forward-secret 0-rtt key exchange. *J. Cryptol.*, 34(2):13, 2021.

DGNW20. Manu Drijvers, Sergey Gorbunov, Gregory Neven, and Hoeteck Wee. Pixel: Multi-signatures for consensus. In Srdjan Capkun and Franziska Roesner, editors, *USENIX Security 2020*, pages 2093–2110. USENIX Association, August 2020.

DH76. Whitfield Diffie and Martin E. Hellman. New directions in cryptography. *IEEE Transactions on Information Theory*, 22(6):644–654, 1976. doi:10.1109/TIT.1976.1055638.

DHP20. Benjamin Dowling, Torben Brandt Hansen, and Kenneth G. Paterson. Many a mickle makes a muckle: A framework for provably quantum-secure hybrid key exchange. In Jintai Ding and Jean-Pierre Tillich, editors, *Post-Quantum Cryptography - 11th International Conference, PQCrypto 2020*, pages 483–502. Springer, Cham, 2020. doi:10.1007/978-3-030-44223-1_26.

DJSS18. David Derler, Tibor Jager, Daniel Slamanig, and Christoph Striecks. Bloom filter encryption and applications to efficient forward-secret 0-RTT key exchange. In Jesper Buus Nielsen and Vincent Rijmen, editors, *EUROCRYPT 2018, Part III*, volume 10822 of *LNCS*, pages 425–455. Springer, Cham, April / May 2018. doi:10.1007/978-3-319-78372-7_14.

DKL+18. David Derler, Stephan Krenn, Thomas Lorünser, Sebastian Ramacher, Daniel Slamanig, and Christoph Striecks. Revisiting proxy re-encryption: Forward secrecy, improved security, and applications. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part I*, volume 10769 of *LNCS*, pages 219–250. Springer, Cham, March 2018. doi:10.1007/978-3-319-76578-5_8.

DRSS21. David Derler, Sebastian Ramacher, Daniel Slamanig, and Christoph Striecks. Fine-grained forward secrecy: Allow-list/deny-list encryption and applications. In Nikita Borisov and Claudia Díaz, editors, *FC 2021, Part II*, volume 12675 of *LNCS*, pages 499–519. Springer, Berlin, Heidelberg, March 2021. doi:10.1007/978-3-662-64331-0_26.

DvOW92. Whitfield Diffie, Paul C. van Oorschot, and Michael J. Wiener. Authentication and authenticated key exchanges. *Des. Codes Cryptogr.*, 2(2):107–125, 1992. doi:10.1007/BF00124891.

Eur19. European Telecommunications Standards Institute. Quantum key distribution (QKD): Protocol and data format of REST-based key delivery API. Standard, European Telecommunications Standards Institute, 2019. URL: https://www.etsi.org/deliver/etsi_gs/QKD/001_099/014/01.01.01_60/gs_qkd014v010101p.pdf.

GHJL17. Felix Günther, Britta Hale, Tibor Jager, and Sebastian Lauer. 0-RTT key exchange with full forward secrecy. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *EUROCRYPT 2017, Part III*, volume 10212 of *LNCS*, pages 519–548. Springer, Cham, April / May 2017. doi:10.1007/978-3-319-56617-7_18.

GHP18. Federico Giacon, Felix Heuer, and Bertram Poettering. KEM combiners. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part I*, volume 10769 of *LNCS*, pages 190–218. Springer, Cham, March 2018. doi:10.1007/978-3-319-76578-5_7.

GPH+24. Lydia Garms, Taofiq K Paraïso, Neil Hanley, Ayesha Khalid, Ciara Rafferty, James Grant, James Newman, Andrew J Shields, Carlos Cid, and Maire O'Neill. Experimental integration of quantum key distribution and post-quantum cryptography in a hybrid quantum-safe cryptosystem. *Advanced Quantum Technologies*, 7(4):2300304, 2024.

Gro21. Jens Groth. Non-interactive distributed key generation and key resharing. Cryptology ePrint Archive, Report 2021/339, 2021. URL: https://eprint.iacr.org/2021/339.

Gün90. Christoph G. Günther. An identity-based key-exchange protocol. In Jean-Jacques Quisquater and Joos Vandewalle, editors, *EUROCRYPT'89*, volume 434 of *LNCS*, pages 29–37. Springer, Berlin, Heidelberg, April 1990. doi:10.1007/3-540-46885-4_5.

HBD+22. Andreas Hülsing, Daniel J. Bernstein, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Panos Kampanakis, Stefan Kölbl, Tanja Lange, Martin M. Lauridsen, Florian Mendel, Ruben Niederhagen, Christian Rechberger, Joost Rijneveld, Peter Schwabe, Jean-Philippe Aumasson, Bas Westerbaan, and Ward Beullens. SPHINCS+. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

HV22. Loïs Huguenin-Dumittan and Serge Vaudenay. On IND-qCCA security in the ROM and its applications - CPA security is sufficient for TLS 1.3. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 613–642. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-07082-2_22.

JLRT23. Paul James, Stephan Laschet, Sebastian Ramacher, and Luca Torresetti. Key management systems for large-scale quantum key distribution networks. In *ARES*, pages 126:1–126:9. ACM, 2023.

Kra03. Hugo Krawczyk. Sigma: The 'sign-and-mac'approach to authenticated diffie-hellman and its use in the ike protocols. In *Annual international cryptology conference*, pages 400–425. Springer, 2003.

KW16. Hugo Krawczyk and Hoeteck Wee. The optls protocol and tls 1.3. In *2016 IEEE European Symposium on Security and Privacy (EuroS&P)*, pages 81–96, 2016. doi:10.1109/EuroSP.2016.18.

LDK+22. Vadim Lyubashevsky, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Peter Schwabe, Gregor Seiler, Damien Stehlé, and Shi Bai. CRYSTALS-DILITHIUM. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

LGM+20. Sebastian Lauer, Kai Gellert, Robert Merget, Tobias Handirk, and Jörg Schwenk. T0RTT: Non-interactive immediate forward-secret single-pass circuit construction. *PoPETs*, 2020(2):336–357, April 2020. doi:10.2478/popets-2020-0030.

LKZC07. Jin Li, Kwangjo Kim, Fangguo Zhang, and Xiaofeng Chen. Aggregate proxy signature and verifiably encrypted proxy signature. In Willy Susilo, Joseph K. Liu, and Yi Mu, editors, *ProvSec 2007*, volume 4784 of *LNCS*, pages 208–217. Springer, Berlin, Heidelberg, November 2007. doi:10.1007/978-3-540-75670-5_15.

Mau93. Ueli M. Maurer. Protocols for secret key agreement by public discussion based on common information. In Ernest F. Brickell, editor, *CRYPTO'92*, volume 740 of *LNCS*, pages 461–470. Springer, Berlin, Heidelberg, August 1993. doi:10.1007/3-540-48071-4_32.

MBO+23. Vicente Martín, Juan Pedro Brito, Laura Ortíz, R. Brito-Méndez, Rafael J. Vicente, J. Saez-Buruaga, A. J. Sebastian, Daniel Gómez Aguado, Marta Irene García Cid, J. Setien, P. Salas, Carmen Escribano, Esther Dopazo, J. Rivas-Moscoso, Antonio Pastor Perales, and Diego R. López. The madrid testbed: QKD SDN control and key management in a production network. In *ICTON*, pages 1–4. IEEE, 2023.

MMP+23. Luciano Maino, Chloe Martindale, Lorenz Panny, Giacomo Pope, and Benjamin Wesolowski. A direct key recovery attack on sidh. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 448–471. Springer, 2023.

MNR+20. Miralem Mehic, Marcin Niemiec, Stefan Rass, Jiajun Ma, Momtchil Peev, Alejandro Aguado, Vicente Martín, Stefan Schauer, Andreas Poppe, Christoph Pacher, and Miroslav Voznák. Quantum key distribution: A networking perspective. *ACM Comput. Surv.*, 53(5):96:1–96:41, 2020. doi:10.1145/3402192.

MSU13. Michele Mosca, Douglas Stebila, and Berkant Ustaoğlu. Quantum key distribution in the classical authenticated key exchange framework. In *Post-Quantum Cryptography: 5th International Workshop, PQCrypto 2013, Limoges, France, June 4-7, 2013. Proceedings 5*, pages 136–154. Springer, 2013.

PFH+22. Thomas Prest, Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Ricosset, Gregor Seiler, William Whyte, and Zhenfei Zhang. FALCON. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

RBBK01. Phillip Rogaway, Mihir Bellare, John Black, and Ted Krovetz. OCB: A block-cipher mode of operation for efficient authenticated encryption. In Michael K. Reiter and Pierangela Samarati, editors, *ACM CCS 2001*, pages 196–205. ACM Press, November 2001. doi:10.1145/501983.502011.

Res18. Eric Rescorla. The transport layer security (TLS) protocol version 1.3. *RFC*, 8446:1–160, 2018. doi:10.17487/RFC8446.

RJ23. Aditya Singh Rawat and Mahabir Prasad Jhanwar. Post-quantum DNSSEC over UDP via qname-based fragmentation. In *SPACE*, volume 14412 of *Lecture Notes in Computer Science*, pages 66–85. Springer, 2023.

Rob23. Damien Robert. Breaking SIDH in polynomial time. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 472–503. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_17.

ROSW24. Eric Rescorla, Kazuho Oku, Nick Sullivan, and Christopher A. Wood. Tls encrypted client hello, 2024. URL: https://datatracker.ietf.org/doc/draft-ietf-tls-esni/.

RRL+19. Thiago R. Raddo, Simon Rommel, Victor Land, Chigo Okonkwo, and Idelfonso Tafur Monroy. Quantum data encryption as a service on demand: Eindhoven QKD network testbed. In *ICTON*, pages 1–5. IEEE, 2019.

RSS23. Paul Rösler, Daniel Slamanig, and Christoph Striecks. Unique-path identity based encryption with applications to strongly secure messaging. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 3–34. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_1.

SAB+22. Peter Schwabe, Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrède Lepoint, Vadim Lyubashevsky, John M. Schanck, Gregor Seiler, Damien Stehlé, and Jintai Ding. CRYSTALS-KYBER. Technical report, National Institute of Standards and Technology, 2022. available at https://csrc.nist.gov/Projects/post-quantum-cryptography/selected-algorithms-2022.

SM16. Douglas Stebila and Michele Mosca. Post-quantum key exchange for the internet and the open quantum safe project. In Roberto Avanzi and Howard M. Heys, editors, *SAC 2016*, volume 10532 of *LNCS*, pages 14–37. Springer, Cham, August 2016. doi:10.1007/978-3-319-69453-5_2.

SS23. Daniel Slamanig and Christoph Striecks. Revisiting updatable encryption: Controlled forward security, constructions and a puncturable perspective. In Guy N. Rothblum and Hoeteck Wee, editors, *TCC 2023, Part II*, volume 14370 of *LNCS*, pages 220–250. Springer, Cham, November / December 2023. doi:10.1007/978-3-031-48618-0_8.

SSW20a. Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 1461–1480. ACM Press, November 2020. doi:10.1145/3372297.3423350.

SSW20b. Peter Schwabe, Douglas Stebila, and Thom Wiggers. Post-quantum TLS without handshake signatures. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*, pages 1461–1480, 2020.

SSW21. Peter Schwabe, Douglas Stebila, and Thom Wiggers. More efficient post-quantum KEMTLS with pre-distributed public keys. In Elisa Bertino, Haya Shulman, and Michael Waidner, editors, *ESORICS 2021, Part I*, volume 12972 of *LNCS*, pages 3–22. Springer, Cham, October 2021. doi:10.1007/978-3-030-88418-5_1.

WZW+21. Liu-Jun Wang, Kai-Yi Zhang, Jia-Yong Wang, Jie Cheng, Yong-Hua Yang, Shi-Biao Tang, Di Yan, Yan-Lin Tang, Zhen Liu, Yu Yu, et al. Experimental authentication of quantum key distribution with post-quantum cryptography. *npj quantum information*, 7(1):67, 2021.